

Lab experiment -02

CSA0605

Design and Analysis of algorithms

1. Find Number of Ways to Move Ball Out of Grid Boundary in Exactly N Steps

Aim

To write a Python program to find the number of ways a ball can move out of an $m \times n$ grid boundary in exactly N steps.

Python Code

```
def findPaths(m, n, N, i, j):  
    dp = {}  
  
    def dfs(x, y, steps):  
        if x < 0 or x >= m or y < 0 or y >= n:  
            return 1 if steps == 0 else 0  
  
        if steps == 0:  
            return 0  
  
        if (x, y, steps) in dp:  
            return dp[(x, y, steps)]  
  
        dp[(x, y, steps)] = (
```

```

        dfs(x + 1, y, steps - 1) +
        dfs(x - 1, y, steps - 1) +
        dfs(x, y + 1, steps - 1) +
        dfs(x, y - 1, steps - 1)
    )
    return dp[(x, y, steps)]

```

```

return dfs(i, j, N)

```

```

m = int(input("Enter rows: "))
n = int(input("Enter columns: "))
N = int(input("Enter number of steps: "))
i = int(input("Enter starting row: "))
j = int(input("Enter starting column: "))

```

```

print("Output:", findPaths(m, n, N, i, j))

```

output

```

Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.

= RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/lab 2 1st.py
Enter rows: 2
Enter columns: 2
Enter number of steps: 2
Enter starting row: 0
Enter starting column: 0
Output: 4
|

```

2. House Robber II (Circular Houses)

Aim

To write a Python program to find the maximum amount of money that can be robbed from circularly arranged houses without robbing adjacent houses.

Python Code

```
def rob_linear(nums):  
    prev = curr = 0  
    for num in nums:  
        prev, curr = curr, max(curr, prev + num)  
    return curr  
  
def rob(nums):  
    if len(nums) == 1:  
        return nums[0]  
    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))  
  
nums = list(map(int, input("Enter house values: ").split()))  
print("Output:", rob(nums))
```

output

```
= RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/lab 2 1st.py =  
Enter house values: 2 3 2  
Output: 3
```

3. Climbing Stairs

Aim

To write a Python program to find the number of distinct ways to climb a staircase when each step can be either 1 or 2 steps.

Python Code

```
def climbStairs(n):
```

```
    if n <= 2:
```

```
        return n
```

```
    a, b = 1, 2
```

```
    for _ in range(3, n + 1):
```

```
        a, b = b, a + b
```

```
    return b
```

```
n = int(input("Enter number of stairs: "))
```

```
print("Output:", climbStairs(n))
```

output

```
= RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/lab 2 1st.py =  
Enter number of stairs: 4  
Output: 5  
|
```

4. Unique Paths in a Grid

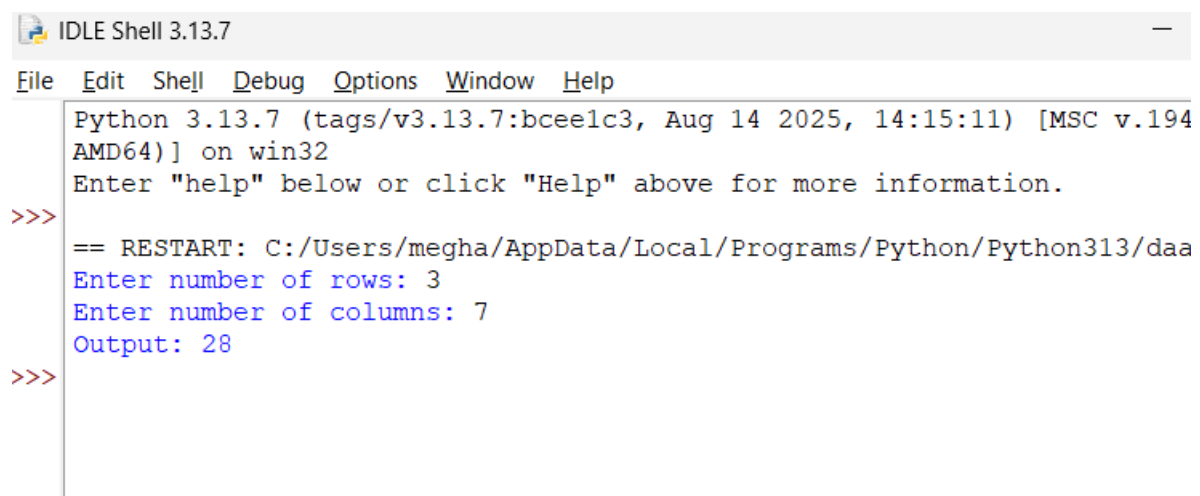
Aim

To write a Python program to find the number of unique paths from the top-left corner to the bottom-right corner of an $m \times n$ grid.

Python Code

```
def uniquePaths(m, n):  
    dp = [[1] * n for _ in range(m)]  
  
    for i in range(1, m):  
        for j in range(1, n):  
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]  
  
    return dp[m - 1][n - 1]  
  
m = int(input("Enter number of rows: "))  
n = int(input("Enter number of columns: "))  
  
print("Output:", uniquePaths(m, n))
```

output



```
IDLE Shell 3.13.7  
File Edit Shell Debug Options Window Help  
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.194  
AMD64] on win32  
Enter "help" below or click "Help" above for more information.  
>>> == RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/daa  
Enter number of rows: 3  
Enter number of columns: 7  
Output: 28  
>>>
```

5. Large Group Positions

Aim

To write a Python program to find all **large groups** (3 or more consecutive identical characters) in a string and return their start and end indices.

Python Code

```
def largeGroupPositions(s):  
    result = []  
    start = 0  
  
    for i in range(1, len(s) + 1):  
        if i == len(s) or s[i] != s[start]:  
            if i - start >= 3:  
                result.append([start, i - 1])  
            start = i  
  
    return result  
  
s = input("Enter the string: ")  
print("Output:", largeGroupPositions(s))
```

output:

```
== RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/daa :  
Enter the string: abbxxxxzy  
Output: [[3, 6]]  
|
```

6. Game of Life

Aim

To write a Python program to generate the **next state** of a Game of Life board based on Conway's rules.

Python Code

```
def gameOfLife(board):  
    m = len(board)  
    n = len(board[0])  
  
    directions = [(-1,-1),(-1,0),(-1,1),  
                  (0,-1),(0,1),  
                  (1,-1),(1,0),(1,1)]  
  
    new = [[0]*n for _ in range(m)]  
  
    for i in range(m):  
        for j in range(n):  
            live = 0  
  
            for dx, dy in directions:  
                x = i + dx  
                y = j + dy  
  
                if 0 <= x < m and 0 <= y < n:  
                    live += board[x][y]  
  
            if board[i][j] == 1:  
                if live == 2 or live == 3:  
                    new[i][j] = 1
```

```
else:
```

```
    if live == 3:
```

```
        new[i][j] = 1
```

```
    return new
```

```
m = int(input("Enter number of rows: "))
```

```
n = int(input("Enter number of columns: "))
```

```
board = []
```

```
print("Enter the board:")
```

```
for i in range(m):
```

```
    board.append(list(map(int, input().split())))
```

```
print("Output:")
```

```
for row in gameOfLife(board):
```

```
    print(row)
```

```
== RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/d
Enter number of rows: 2
Enter number of columns: 2
Enter the board:
1 1
1 0
Output:
[1, 1]
[1, 1]
```


7. Champagne Tower

Aim

To write a Python program to determine how full a specified glass is after pouring a given number of cups of champagne into a tower.

Python Code

```
def champagneTower(poured, query_row, query_glass):  
    dp = [[0.0] * 101 for _ in range(101)]  
    dp[0][0] = poured  
  
    for r in range(query_row + 1):  
        for c in range(r + 1):  
            overflow = (dp[r][c] - 1.0) / 2.0  
            if overflow > 0:  
                dp[r + 1][c] += overflow  
                dp[r + 1][c + 1] += overflow  
  
    return min(1.0, dp[query_row][query_glass])  
  
poured = int(input("Enter poured cups: "))  
query_row = int(input("Enter query row: "))  
query_glass = int(input("Enter query glass: "))  
  
print("Output:", "{:.5f}".format(champagneTower(poured, query_row, query_glass)))
```

output

```
== RESTART: C:/Users/megha/AppData/Local/Programs/Python/Python313/daa 1  
Enter poured cups: 2  
Enter query row: 1  
Enter query glass: 1  
Output: 0.50000
```